

Lab Program 2: Candidate Elimination Algorithm

```
import csv
```

```
# -----
```

```
# Candidate Elimination Algorithm
```

```
# -----
```

```
# Read training data from CSV file
```

```
with open("training_data.csv", "r") as file:
```

```
    reader = csv.reader(file)
```

```
    data = list(reader)
```

```
# First row contains attribute names
```

```
attributes = data[0]
```

```
# Remaining rows contain training examples
```

```
training_data = data[1:]
```

```
# Number of attributes
```

```
num_attributes = len(attributes) - 1
```

```
# -----
```

```
# Initialize Specific Boundary
```

```
# -----
```

```
S = ['0'] * num_attributes
```

```
# -----
```

```
# Initialize General Boundary
```

```
# -----
```

```
G = [['?'] * num_attributes]
```

```
# -----
```

```
# Display Training Data
```

```
# -----
```

```
print("Training Data:")
```

```
print("-" * 70)
```

```
for row in training_data:
```

```
    print(row)
```

```
print("\n")
```

```
# -----
```

```
# Candidate Elimination
```

```
# -----
```

```
for i, row in enumerate(training_data):
```

```
    instance = row[:-1]
```

```
    target = row[-1]
```

```
    print("Training Example", i + 1)
```

```
    print("Instance :", instance)
```

```
    print("Target  :", target)
```

```
# -----
```

```
# Positive Example
```

```
# -----
```

```
if target == "Yes":
```



```
# If S is empty, initialize it
if S == ['0'] * num_attributes:
    S = instance.copy()

else:

    # Generalize S where necessary
    for j in range(num_attributes):

        if S[j] != instance[j]:
            S[j] = '?'

# Remove hypotheses from G
# that are inconsistent with S
new_G = []

for g in G:

    consistent = True

    for j in range(num_attributes):
```

```
if g[j] != '?' and g[j] != S[j]:
```

```
    consistent = False
```

```
    break
```

```
if consistent:
```

```
    new_G.append(g)
```

```
G = new_G
```

```
# -----
```

```
# Negative Example
```

```
# -----
```

```
else:
```

```
    new_G = []
```

```
for g in G:
```

```
    # Check whether G hypothesis covers
```

```
    # the negative example
```

```
covers = True
```

```
for j in range(num_attributes):
```

```
    if g[j] != '?' and g[j] != instance[j]:
```

```
        covers = False
```

```
        break
```

```
if covers:
```

```
    # Generate minimal specializations
```

```
    for j in range(num_attributes):
```

```
        if g[j] == '?':
```

```
            if S[j] != instance[j]:
```

```
                new_hypothesis = g.copy()
```

```
                new_hypothesis[j] = S[j]
```

```
                # Add only if it is not already present
```



```
        if new_hypothesis not in new_G:
            new_G.append(new_hypothesis)

    else:
        new_G.append(g)

    G = new_G

# -----
# Display Boundaries
# -----

print("Specific Boundary (S):")
print(S)

print("General Boundary (G):")

for hypothesis in G:
    print(hypothesis)

print("-" * 70)
```

Final Result

```
print("\nFINAL RESULT")
```

```
print("=" * 70)
```

```
print("\nSpecific Boundary (S):")
```

```
print(S)
```

```
print("\nGeneral Boundary (G):")
```

```
for hypothesis in G:
```

```
    print(hypothesis)
```